



PDF Download
3719545.3759428.pdf
16 January 2026
Total Citations: 0
Total Downloads: 309

Latest updates: <https://dl.acm.org/doi/10.1145/3719545.3759428>

RESEARCH-ARTICLE

Multi-agent Multi-game Entity Transformer: Towards Generalist Models in MARL

RUNDONG WANG, Nanyang Technological University, Singapore City, Singapore

WEIXUAN WANG, Tencent, Shenzhen, Guangdong, China

XIANHAN ZENG, Tencent, Shenzhen, Guangdong, China

LIANG WANG, Tencent, Shenzhen, Guangdong, China

ZHENGJIE LIANG, Tencent, Shenzhen, Guangdong, China

YIMING GAO, Tencent, Shenzhen, Guangdong, China

[View all](#)

Open Access Support provided by:

[Carleton University](#)

[Tencent](#)

[Nanyang Technological University](#)

Published: 18 December 2024

[Citation in BibTeX format](#)

DAI '24: 6th International Conference on Distributed Artificial Intelligences
December 18 - 22, 2024
Singapore, Singapore

Multi-agent Multi-game Entity Transformer: Towards Generalist Models in MARL

Rundong Wang
Nanyang Technological University
Singapore, Singapore
RUNDONG001@e.ntu.edu.sg

Liang Wang
Tencent
Shenzhen, China
enginewang@tencent.com

Feiyu Liu
Tencent
Shenzhen, China
marsxliu@tencent.com

Qiang Fu
Tencent
Shenzhen, China
leonfu@tencent.com

Longtao Zheng
Nanyang Technological University
Singapore, Singapore
longtao001@e.ntu.edu.sg

Weixuan Wang
Tencent
Shenzhen, China
waihincw@tencent.com

Zhengjie Liang
Tencent
Shenzhen, China
leolian@tencent.com

Siqin Li
Tencent
Shenzhen, China
gracesqli@tencent.com

Wei Yang
Tencent
Shenzhen, China
willyang@tencent.com

Zinovi Rabinovich
Carleton University
Ottawa, Canada
zr@zinovi.net

Xianhan Zeng
Tencent
Shenzhen, China
ritzeng@tencent.com

Yiming Gao
Tencent
Shenzhen, China
yatminggao@tencent.com

Xianliang Wang
Tencent
Shenzhen, China
lemxlwang@tencent.com

Lanxiao Huang
Tencent
Shenzhen, China
jackiehuang@tencent.com

Bo An
Nanyang Technological University
Singapore, Singapore
boancqu@gmail.com

Abstract

Building large-scale generalist pre-trained models for many tasks is becoming an emerging and potential direction in reinforcement learning (RL). Research such as Gato and Multi-Game Decision Transformer have displayed outstanding performance and generalization capabilities on many games and domains. However, there exists a research blank about developing highly capable and generalist models in multi-agent RL (MARL), which can substantially accelerate progress toward general AI. To fill this gap, we propose Multi-Agent multi-Game ENtity TrAnsformer (MAGENTA) from the entity perspective as orthogonal research to previous time-sequential modeling. Specifically, to deal with different state/observation spaces in different games, we analogize games as languages by aligning one single game to one single language, thus training different "tokenizers" and a shared transformer for various games. The feature inputs are split according to different entities and tokenized in the same continuous space. Then, two types of transformer-based models are proposed as permutation-invariant architectures to deal with various numbers of entities and capture the attention of different entities. MAGENTA is trained on

Honor of Kings, Starcraft II micromanagement, and Neural MMO with a single set of transformer weights. Extensive experiments show that MAGENTA can play games across various categories with arbitrary numbers of agents and increase the efficiency of fine-tuning in new games and scenarios by 50%-100%. See our project page at <https://sites.google.com/view/rl-magenta>.

Keywords

Multi-agent Reinforcement Learning, Generalist Model, Pretraining

ACM Reference Format:

Rundong Wang, Weixuan Wang, Xianhan Zeng, Liang Wang, Zhengjie Liang, Yiming Gao, Feiyu Liu, Siqin Li, Xianliang Wang, Qiang Fu, Wei Yang, Lanxiao Huang, Longtao Zheng, Zinovi Rabinovich, and Bo An. 2024. Multi-agent Multi-game Entity Transformer: Towards Generalist Models in MARL. In *Sixth International Conference on Distributed Artificial Intelligences (DAI '24)*, December 18–22, 2024, Singapore, Singapore. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3719545.3759428>

1 Introduction

In recent years, transformer-based models, as a solution to build large-scale generalist models, have made substantial progress in natural language processing [4, 7], computer vision [1, 8], graph learning [20, 29]. Furthermore, they are showing their potential in reinforcement learning (RL) [14, 19, 26] by modeling and solving sequential problems.



This work is licensed under a Creative Commons Attribution 4.0 International License. *DAI '24, Singapore, Singapore*
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1007-0/24/12
<https://doi.org/10.1145/3719545.3759428>

However, there are a few inherent challenges in building large-scale general RL agents. First, when the training environment and the test environment are the same, RL is inclined to overfit the training environments, while lacking generalizability to unknown environments. As a result, a model is often needed to be re-trained from scratch for a new task. Second, it is challenging for a single model to adapt for various environments with differences in the number of agents, states, observations, actions, and dynamics. Third, training from scratch normally suffers from expensive computational costs, especially for large-scale RL. For example, AlphaStar requires 16 TPUs to train for 14 days, and Honor of King (HoK) requires 19,600 core CPUs and 168 V100 GPUs to train for nearly half a month. Thus, building a general, reusable, and efficient RL model has been an increasingly important task for both industrial and non-industrial research.

To this end, we investigate whether a single model, with a single set of parameters, can be trained by playing multiple multi-agent games in an online manner, which is a blank in current research after Gato [19] and MGDT [14]. We consider training on Honor of Kings (HoK), Starcraft II micromangement (SMAC), and Neural MMO (NMMO), informally asking:

Can models learn some general knowledge about games across various categories?

In this paper, we answer this question by proposing Multi-Agent multi-Game ENtity TrAnsformer (MAGENTA). We consider this problem as a few-shot transfer learning with the hypothesis, that a single model is capable of playing many games and can be adapted to never-seen-before games or scenarios with fine-tuning. For interpretability, we align the design of MAGENTA with the Entity Component System (ECS) architectural pattern of video games and the multilingual transformer. Specifically, we treat different games as different languages by aligning one single game to one single language as shown in Fig.1. Different languages have different tokenizers, so do games. Also, we expect that the learned representations in different games are similar to the word2vec in NLP. In this case, our transformer can be viewed as a multilingual transformer to capture the common knowledge among different games. We split the feature input according to different entities and tokenize the features in the same continuous space. Unlike existing applications of the transformer as a causal time-sequential model, the transformer in MAGENTA serves as a permutation-invariant architecture to attend to the features of different entities. We propose two types of transformer architecture, Encoder-Pooling (EP) and Encoder-Decoder (ED), to build permutation-invariant models. In this way, the output of the model should not change under any permutation of the elements in the input. Lastly, we show the training scheme of MAGENTA about how to train and transfer the pre-trained model.

As a step towards developing a general model in RL/MARL, our contributions are threefold: First, by aligning games and languages, we show that it is possible to train a single generalist agent to act across multiple environments in an online manner. Second, we present permutation-invariant models for adapting various numbers of agents in different games. And third, we find that MAGENTA can achieve rapid fine-tuning in an online fashion to different scenarios within a single game, to a new type of game,



Figure 1: Align games as languages. (Above) Tokenizing in one game and one language. (Below) Tokenize different observations from different games and different sentences from different languages.

and to different numbers of agents. Furthermore, we release the pre-trained models and code to encourage further research in this direction.

2 Preliminary

2.1 Reinforcement Learning and Multi-agent RL

An RL problem is generally studied as a Markov decision process (MDP) [2], defined by the tuple: $\text{MDP} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, T)$, where $\mathcal{S} \subseteq \mathbb{R}^n$ is an n -dimensional state space, $\mathcal{A} \subseteq \mathbb{R}^m$ an m -dimensional action space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_+$ a transition probability function, $r : \mathcal{S} \rightarrow \mathbb{R}$ a bounded reward function, $\gamma \in (0, 1]$ a discount factor and T a time horizon. In MDP, an agent receives the current state $s_t \in \mathcal{S}$ from the environment and performs an action $a_t \in \mathcal{A}$ defined by a policy $\pi_\theta : \mathcal{S} \rightarrow \mathcal{A}$ parameterized by θ . The objective of the agent is to learn an optimal policy: $\pi_{\theta^*} := \arg\max_{\pi_\theta} \mathbb{E}_{\pi_\theta} [\sum_{i=0}^T \gamma^i r_{t+i} | s_t = s]$. A MARL problem is formulated as a *decentralized partially observable Markov decision process* (Dec-POMDP) [3], which is described as a tuple $\langle n, \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{O}, \Omega, \gamma \rangle$, where n represents the number of agents. $\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}$ are all global versions of those in MDP. $\mathcal{O} = \{O_i\}_{i=1, \dots, n}$ denotes the space of observations of all agents. Each agent i receives a private observation $o_i \in O_i$ according to the observation function $\Omega(s, i) : \mathcal{S} \rightarrow O_i$.¹

2.2 Attention Mechanism in Transformer

One of the most essential components of Transformer [25] is the attention mechanism, which captures the interrelationship of input sequences. Assume that we have n query vectors (corresponding to a set with n elements) each with dimension $d_q : Q \in \mathbb{R}^{n \times d_q}$. The attention function is written as $\text{Attention}(Q, K, V) = \omega(QK^T)V$, which maps queries Q to outputs using n_v key-value pairs $K \in \mathbb{R}^{n_v \times d_k}, V \in \mathbb{R}^{n_v \times d_v}$. The pairwise dot product $QK^T \in \mathbb{R}^{n \times n_v}$ measures how similar each pair of query and key vectors is, with weights computed with an activation function ω . The output $\omega(QK^T)V \in \mathbb{R}^{n \times d_v}$ is a weighted sum of V where a value gains more weight if its corresponding key has a larger dot product with

¹We present this paper in the scope of MARL, and abuse the term "feature" an alternative of observation.

the query. Furthermore, self-attention refers to cases where Q, K, V share the same set of parameters. Multi-head attention is an extension of attention by computing h attention functions simultaneously and outputting a linear transformation of the concatenation of all attention outputs.

3 Related Work

3.1 Transformer in RL

Recently, transformer-based models have been shown to be a powerful tool for RL to capture global dependencies or model long-term sequence. The architecture of MAGENTA is closely related to AlphaStar [11] and OpenAI Five [17], which use transformer-based models to embed unit features for each player. Another perspective of using transformer-based models is to model an RL problem as a sequence prediction problem that models trajectories autoregressively, such as Decision Transformer (DT) [6], Trajectory Transformer (TT) [12], GATO [19], and Multi-Game DT (MGDT) [14]. In addition, Multi-Agent transformer (MAT) [26] considers the MARL problem as a sequence prediction problem that generates the optimal action of each agent sequentially.

However, these transformer-based models study the generality with *trajectory sequences* in *single agent* scenarios [14, 19] or focus on a *single task* with the sequences [6, 11, 12, 17, 26]. In addition, they use causal transformer decoders, which use causal masking to discard future information. MAGENTA fills the current research blank by using a transformer model without positional encoding and exploring generality with entity sequences in multi-agent scenarios. Note that models such as DT, TT, and MGDT utilize the sequence along the time dimension, while MAGENTA along the entity dimension, so MAGENTA is orthogonal to time trajectory sequence modeling methods by viewing MAGENTA as a tokenizer in these methods. We summarize these related works as shown in Table 1.

3.2 Reuse Trained RL Models

Another related research direction is to reuse trained RL models, which is an important subfield of RL. Some approaches directly use the trained models. For example, to avoid restarting from scratch after changes in code and environment, OpenAI Five [17] used the "surgery" approach to convert a trained model to certain larger architectures with customized weight initialization. MGDT [14] and Gato [19] follow the manner of few-shot transfer learning by using a single set of weights to play many games or handle tasks in multiple domains simultaneously. Furthermore, some approaches implicitly exploit trained models, such as behavior cloning [9, 10], distillation [18, 21], and the use of the teacher-student framework [5, 22, 23]. Considering that all these methods are based on well-pretrained model weights, MAGENTA follows the fashion of direct usage and provides our code and trained agents. Interested researchers can use the provided models for further exploration of how to reuse such models.

4 Multi-agent Multi-game Entity Transformer

In building generic multi-agent AI for different games, the following challenges are encountered: (1) different feature inputs (i.e., different observations), different action spaces, and different rewards

across different classes of games; (2) the number of entities varies across different classes of games.

Motivated by multilingual models [7, 15, 27] in the NLP community, we treat different games as different languages to deal with different inputs of features. Although words or sentences in different languages have different representations, such as "*I love you*" in English, and "*Je t'aime*" in French; however, the meaning behind those words is the same. We also find the analogy in the ECS game design pattern. Games are designed based on physical rules in the world. Basically, entities in most games can move, collide, and even attack. The components or attributes of these entities are changing according to the "virtual rules", which are mimics of the physical rules. The RL policy can be considered as a neural network system to approximate such rules. So we divide such neural network systems into three parts: tokenizer, transformer, and output. Tokenizers are trained for different games, just as they are trained for different languages in NLP. Transformer is a multi-game transformer, as an analogy to the multilingual transformer. While output are finetuned according to the down-streaming tasks.

4.1 Games as Languages

Tokenizer is used to obtain initial embeddings of words in the NLP community. When we treat games as languages, different features of entities can be analogized to different "words". Such "words" are typically composed of images and vectors. Therefore, we need to divide the features into "words" and then map them into the same *continuous* space.

A good tokenization for different games should be feature-space-agnostic. Different games have different features, such as images or vectors. Furthermore, the features of different entities have different shapes, dimensions, and meanings. Therefore, good tokenization is supposed to handle different feature spaces and split the entire feature into individual elements, such as the tokenizer in NLP splits sentences into words.

We categorize the feature input into two types: images and vectors. When playing games, we can have a screenshot as an image. Typically, we call them frames as shown in [16]. The numerical attributes of the entities can be vectors. For example, a vector composed of the health point, the mana point, the speed, the attack, and the defense, etc. As shown in Fig.2, to process images, we first use convolution neural networks to extract the latent representation and then linearly project the representation into the embedding space. For processing vectors, we divide the vectors into many inputs that correspond to different entities. Then we use a fully connected neural network to get tokens for different entities. The same types of entities share the same parameters in neural networks.

We chose our tokenization scheme with simplicity in mind, but many other schemes are possible. For example, more sophisticated methods, such as VAE [13] can be used to learn a more effective token representation.

4.2 Transformer in MAGENTA

In many game scenarios, the number of entities to be modeled changes over time (e.g., deaths of soldiers in Starcraft II, the revival of enemies in Neural MMO), and although some empty spaces can be set aside in advance from a network compatibility perspective,

Table 1: Related Work on Transformers in RL

Method	Sequence	Data	Agent	Task	Transfer	Games/Domains
DT, TT	trajectory	offline	single	single	No	Atari, OpenAI Gym, Key-to-Door
Gato	trajectory	offline	single	multi	No	Atari, caption, chat, robotics, etc.
MGDT	trajectory	offline	single	multi	Yes	Atari
online DT	trajectory	online	single	single	Yes	OpenAI Gym
MAT	agent	online	multi	single	No	Starcraft II micromanagement
AlphaStar	entity	online	multi	single	No	Starcraft II full game
OpenAI Five	entity	online	multi	single	No	Dota 2
MAGENTA	entity	online	multi	multi	Yes	HoK, Neural MMO, Starcraft II micromanagement

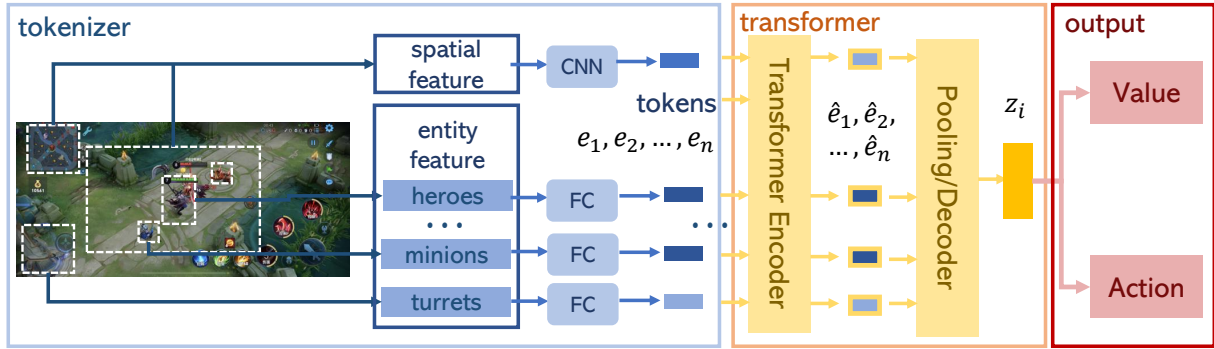


Figure 2: Overall framework of MAGENTA. (Left) An example tokenizer for HoK. Spatial features are as images and entity features as vectors. **(Mid)** The transformer architecture in MAGENTA. We denote the token representations of entities by $(e^1, \dots, e^n)$ and the output of the attentive features by $(\hat{e}^1, \dots, \hat{e}^n)$. We unify z_i from different architectures as the permutation-invariant representation. **(Right)** The value head and policy head for decision making.

these spaces are not always suitable for new entities. In addition, the order of the entities should not affect the RL model training. Unlike the order of words in sentences, the order dependence in RL leads to an inability to handle the input of a varying number of entities.

A model for this issue should satisfy two critical requirements. First, the model should be able to process input of any size. The number of input tokens depends on the number of entities. Second, it should be order-agnostic or permutation-invariant. The output of the model should not change under any permutation of the elements in the input. Classical feedforward neural networks violate both requirements and RNNs are sensitive to input order. To this end, we use the transformer architecture as the core of MAGENTA. We modify the vanilla transformer architecture by removing the positional encoding to satisfy these two requirements.

Furthermore, to force our model to order agnostic, we design two types of transformer architecture, as shown in Figure 2: Encoder-Pooling (EP) and Encoder-Decoder (ED). Detailed description can be found in Appendix. The encoder is used to obtain the mapping between the token representations of entities $(e^1, \dots, e^n)$ and the output of the attentive features $(\hat{e}^1, \dots, \hat{e}^n)$, where n is the number of all entities. The key idea is that each agent can only affect the related entities, and thus should pay more attention to them.

For EP, we use the pooling function to post-process the embeddings. A network that performs pooling over embeddings extracted from the elements of a set. More formally, $\text{net}(\{\hat{e}^1, \dots, \hat{e}^n\}) = \rho(\text{pool}(\{\phi(\hat{e}^1), \dots, \phi(\hat{e}^n)\}))$. Here, operations like mean, summation, max, or similar can be used as the pooling function to aggregate the embeddings. Zaheer et al. have proven that all permutation-invariant functions can be represented as this inline equation when pool is the sum operator and ϕ, ρ any continuous functions, thus satisfying the permutation-invariance requirement.

For ED, we design a decoder to post-process the embeddings. Specifically, instead of using self-attention, the decoder uses the original attention mechanism and outputs $\text{Attention}(Q, K, V) \in \mathbb{R}^{1 \times d_v}$, where Q is the embedding $\hat{e}^i$ of the current agent i (an agent is also an entity), while K, V are the embeddings of all entities. In this way, the output of ED is only one single vector that does not change under any permutation of the elements in the input.

4.3 Training and Implementation Details

Training scheme. We focus on MARL scenarios and choose three representative environments from the industrial and academic domains: Honor of Kings (HoK), Neural MMO (NMMO), and Starcraft II micromanagement (SMAC). We design our three-stage training scheme as shown in Fig.4.

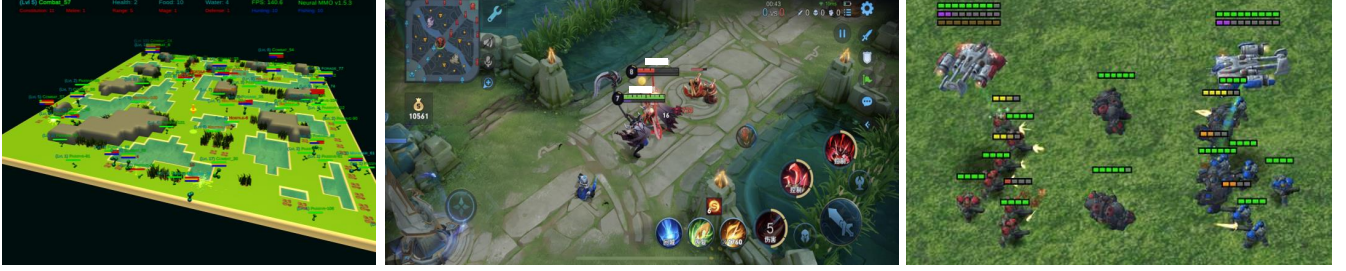


Figure 3: Games where MAGENTA trains. (Left) Neural MMO, a massively multiagent game environment provided by OpenAI; (Mid) Honor of Kings, the best-selling mobile game worldwide; (Right) Starcraft II micromanagement, the most popular MARL research environment. See detailed descriptions of these games in the Appendix.

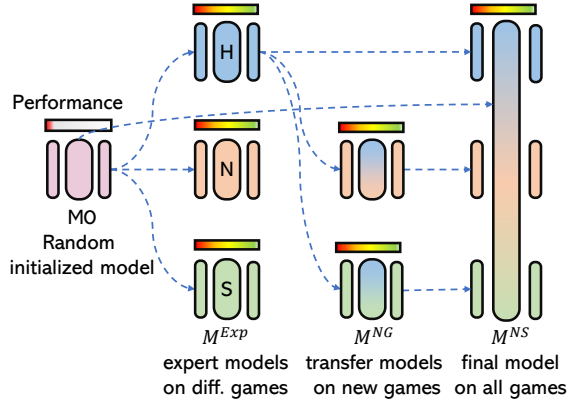


Figure 4: The training scheme of MAGENTA. The dotted lines represent transfers to a new game or a new scenario. Each cell represents a tokenizer, transformer, and output, respectively. Different colors represent different games.

Stage 1. From the random initialized model M^0 , we train expert models with MAGENTA on each individual game and obtain expert models M_H^{Exp} , M_N^{Exp} , M_S^{Exp} in different games, where H represents HoK, N represents NMMO, S represents SMAC.

Stage 2. We transfer M_H^{Exp} to new games and get $M_{H,N}^{NG}$, $M_{H,S}^{NG}$. The tokenizer and output are trained from scratch in the new games. Note that HoK, with a state and action space of magnitude 10^{20000} [28], is much more complex. So we transfer the trained HoK transformer to NMMO and SMAC.

Stage 3. We start from M^0 and train the transformer $M^{NS} = M_{H,N,S}^{NS}$ in selected scenarios in these three environments. The tokenizer and output are from M^{NG} . Then we check whether M^{NS} can handle never-seen-before scenarios in these games with few-shot transfer ²

Policy updates. In order to avoid training instability in our large-scale distributed environment, we use the dual-clip PPO method for each agent. Unlike the original algorithm, we introduce the policy $\pi_\theta(a_i | o_i)$ and the estimate of advantages $\hat{A}_t(a_i, o_i)$. When $\pi_\theta(a_i | o_i) \gg \pi_{\theta_{old}}(a_i | o_i)$ and $\hat{A}_t < 0$, the ratio $r_t(\theta) =$

²We follow MAT [26]’s setting to treat 1%, 5%, 10% of 10M-shot (100k, 500k, 1M-shot) as few-shot in the SMAC/NMMO environment.

$\frac{\pi_\theta(a_i | o_i)}{\pi_{\theta_{old}}(a_i | o_i)}$ is huge, causing the large and unbounded variance since $r_t(\theta)\hat{A}_t \ll 0$. Dual-clip PPO introduces another clipping parameter c that indicates the lower bound when $\hat{A}_t < 0$. The new objective is defined as:

$$L^{\text{policy}}(\theta) = \hat{\mathbb{E}}_t \left[\max \left(c\hat{A}_t, \min \left(\text{clip}(r_t(\theta), 1 - \tau, 1 + \tau)\hat{A}_t, r_t(\theta)\hat{A}_t \right) \right) \right] \quad (1)$$

where τ is the original clip parameter in PPO.

Value updates. We use the multi-head value mechanism for different reward sources. Specifically, the values of achieving one reward are regarded as one head. Therefore, the value loss is defined as:

$$L^{\text{value}}(\theta) = \hat{\mathbb{E}}_t \left[\sum_{\text{head } k} \left(R_t^k - \hat{V}_t^k \right)^2 \right] \quad (2)$$

where w_k is the learnable weight of the k -th head and $V_t^k(o_i)$ is the k -th value.

5 Experiments

We answer the following research questions. **Q1:** How is the performance of the entity transformer in a single game? (Sec. 5.2) **Q2:** How effective is MAGENTA in transfer to **new games**? (Sec. 5.3) **Q3:** How effective is MAGENTA in transfer to **new scenarios**? (Sec. 5.4) **Q4:** How does MAGENTA **scale with model size**? (Sec. 5.5) **Q5:** What does MAGENTA attend to? (See Appendix 5.6 due to page limits.) **Q6:** What does the learned embedding look like? (See Appendix 5.7)

5.1 Experimental Setup

Models used in the experiments. Magenta is trained in three environments: Honor of King, Starcraft II micromanagement SMAC, and Neural MMO. We summarize the training set with the abbreviation H, S, and N to represent the three games in the following table. We list the different versions of MAGENTA and the corresponding transfer experiments in Table 2.

Training and fine-tuning. We train MAGENTA models on Hok with an ADAM optimizer of a $2 \cdot 10^{-4}$ learning rate, $\beta_1 = 0.9$ and $\beta_2 = 0.999$, without weight decay, gradient clip 0.5, and batch size 2048. And we train MAGENTA on SMAC and NMMO with an ADAM optimizer of $1 \cdot 10^{-4}$ learning rate, $\beta_1 = 0.9$ and $\beta_2 = 0.999$,

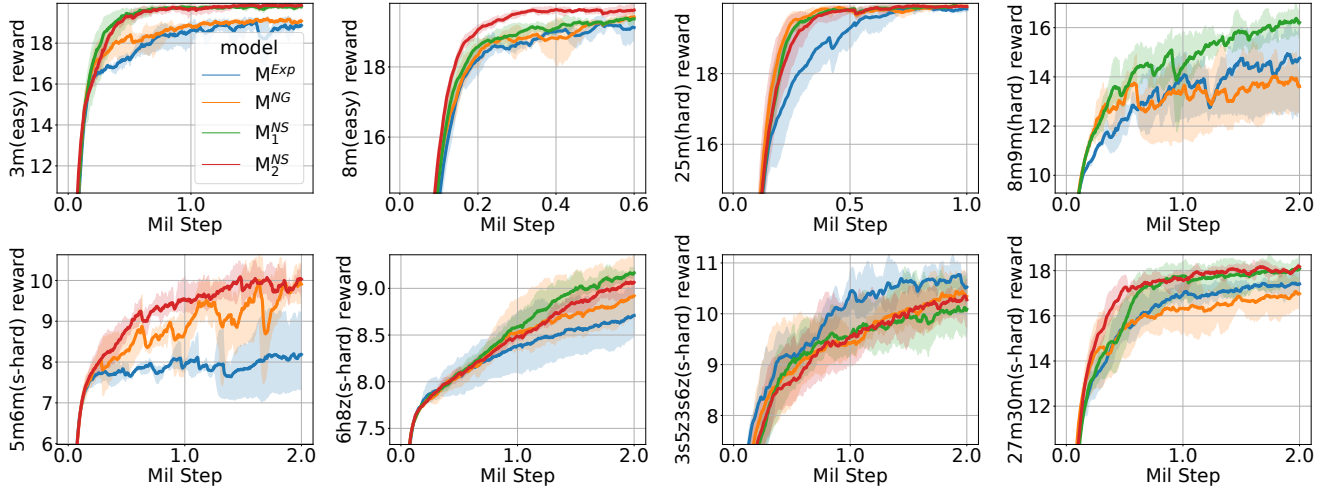


Figure 5: Transfer performance in SMAC. The y-axis is named with “X”m(“hardness”), which represents the different tasks in SMAC.

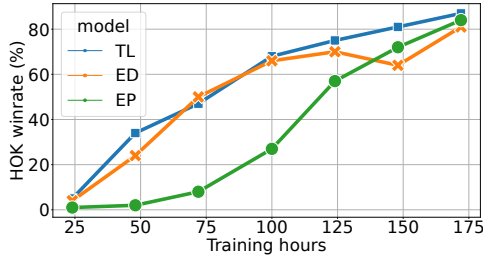


Figure 6: The performance of different variants of MAGENTA in HoK 3v3³.

Table 2: Transfer experimental settings. A->B means training on A and transferring to B.

	EP	ED	TL
M^{Exp}	H,S,N	H,S,N	H
M^{NG}	H->	H->	-
(new games)	S,N	S,N	-
M^{GS}	H+S+N->	H+S+N->	-
(new scenarios)	S,N	S,N	-

no weight decay, gradient clip 0.3, and batch size 4096. Detailed hyperparameters are described in the Appendix ??.

Metrics. We measure the performance of MAGENTA in HoK by the win rate and in SMAC/NMMO by cumulative rewards. Due to prohibitively long training times, we only evaluated one training seed in HoK and three training seeds in SMAC/NMMO.

5.2 How is the performance of the entity transformer?

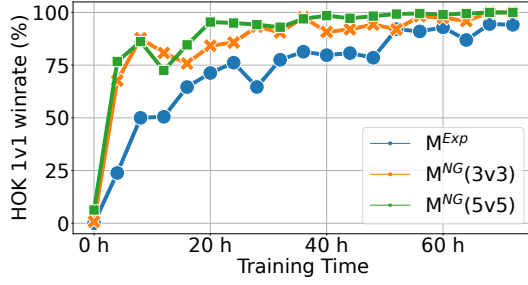
In this subsection, we mainly evaluate M^{Exp} of MAGENTA in HoK 3v3 scenario. We trained 3 different variants of MAGENTA: EP, ED, TL (EP followed by an LSTM network), and train FC+LSTM (FC layers as replacement of the transformer) as the opponent to calculate the win rate with three variants. All variants have similar model sizes. We introduce the TL model to show that MAGENTA is orthogonal to the time-sequential modeling methods. At each point, two models with the same training time compete with each other.

In Fig.6, we can see that after 160 hours of training, all variants of MAGENTA outperform the FC+LSTM model. It shows the benefit of using the transformer architecture. An interesting finding is that, at the beginning, MAGENTA hardly won the FC+LSTM model. The transformer can increase model capacity and expressiveness while needing more training data compared to the FC model.

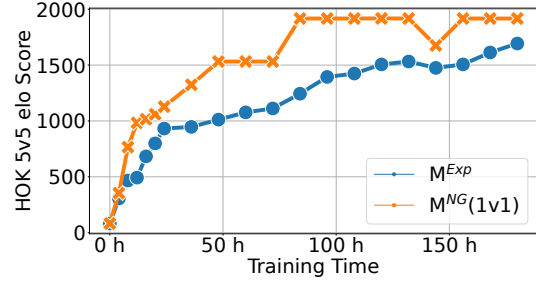
We also provide a comparison to existing single-game baselines. Since Magenta is a multi-game multi-agent method focusing on getting good knowledge transfer from training on multiple games, we select the optimal performance of other single-game MARL algorithms in every single game as the algorithmic baselines. We report IPPO in HoK and NMMO as the single game baseline, since HoK, and NMMO provide individual rewards while QMIX is trained based on a global reward assumption. Then we report the win rate of trained QMIX and VMIX in SMAC as the single-game baseline in Table.3 and 4.

5.3 How effective are MAGENTA at transfer to new games?

We want to evaluate whether MAGENTA can adapt the novel games. To do this, we transfer the trained HoK transformer to NMMO and SMAC. We show the training process of M^{Exp} , M^{NG} and M^{NS} . In Fig.5, we show the result of the transfer to SMAC. Before this



(a) From 5v5 and 3v3 to 1v1.



(b) From 1v1 to 5v5.

Figure 7: The transfer in HoK. The win rate is against the FC+LSTM model.

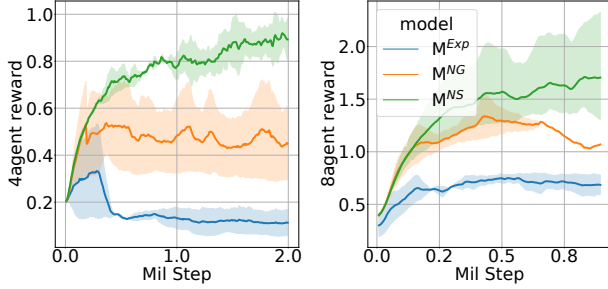


Figure 8: Transfer performance in NMMO.

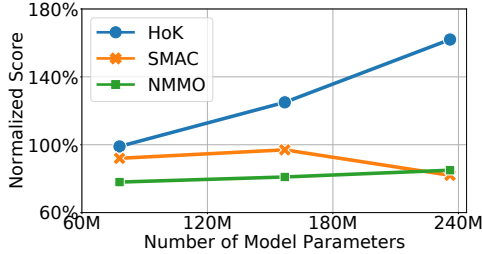


Figure 9: Normalized scores for all training games with different model sizes.

transfer, M^{NG} is already trained in HoK, M_1^{NS} in HoK, SMAC 5m_vs_6m scenario, NMMO 8 agent scenario, and M_2^{NS} in HoK, SMAC 8m_vs_9m, NMMO 8 agent scenario. For M^{NG} , the tokenizer and the output are trained from scratch with the pre-trained transformer. For M^{NS} , all modules in MAGENTA are pre-trained. In this subsection, we focus on M^{Exp} and M^{NG} , since M^{NG} is trained in the new games. We can see that M^{NG} can outperform M^{Exp} in most SMAC scenarios, including easy, hard, and super hard scenarios. In Fig.8, we show the result of the transfer to NMMO. We show the performance in the 4-agent scenario and the 8-agent scenario. We can clearly see that M^{NG} can outperform M^{Exp} . The results verify

SMAC Scenarios	Difficulty	Finetuned-QMIX (Value-based)	VMIX (Policy-based)	Magenta (Policy-based)
3m	Easy	100%	100%	100%
8m	Easy	100%	100%	100%
25m	Easy	100%	100%	100%
8m_vs_9m	Hard	84%	75%	100%
5m_vs_6m	S-Hard	73%	9%	50%
6h_vs_8z	S-Hard	84%	80%	88%
3s5z_vs_3s6z	S-Hard	65%	56%	74%
27m vs 30m	S-Hard	95%	93%	97%

Table 3: Comparison to existing single-game (SMAC) baselines.

IPPO Scenarios	IPPO (Policy-based)	Magenta (Policy-based)
4agent playground	0.32	0.91
8agent playground	1.19	1.68

Table 4: Comparison to existing single-game (nmno) baselines.

our hypothesis that pre-training in other games should indeed help with the rapid learning of a new game.

5.4 How effective is MAGENTA in transfer to new scenarios?

We devise our own evaluation setup by transferring to the never-seen-before scenarios in HoK, SMAC, and NMMO. HoK has three scenarios: 1v1, 3v3, and 5v5. In HoK, we have two experiments to answer whether we can transfer to more or fewer agents.

In Fig.7a, we show the training performance of model M_{HoK}^{Exp} , M_{1v1}^{NG} , M_{3v3}^{NG} and M_{5v5}^{NG} . The result shows that the models transferred from 3v3 and 5v5 to 1v1 outperform the model trained in 1v1 from scratch. And there is a slight performance difference between M_{3v3}^{NG} and M_{5v5}^{NG} . In Fig.7b, the model transferred from 1v1 to 5v5 outperforms the model trained in 5v5 from scratch. The experiments show that MAGENTA learned some transferable knowledge that can be used in the never-seen-before scenarios, even in the

⁴We omit other games in the subscript.

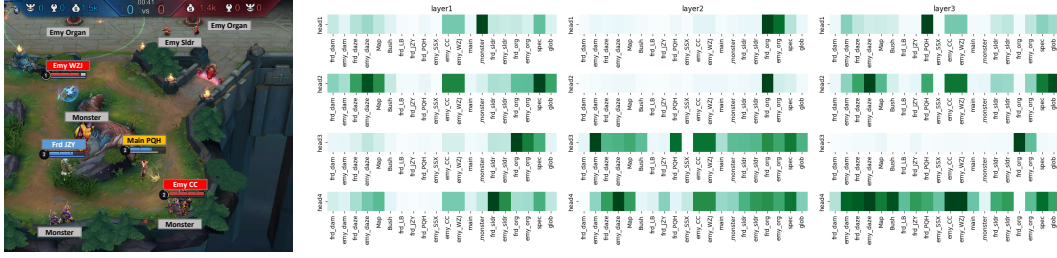


Figure 10: The attention of the entity transformer on 3v3



Figure 11: The visualization of attention weights in SMAC.

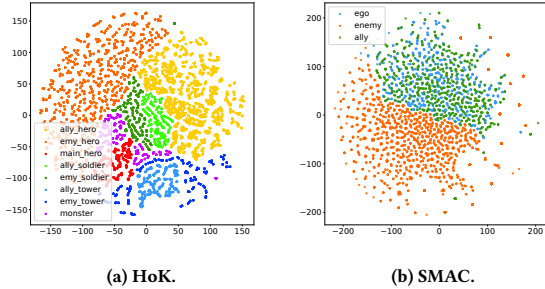


Figure 12: The visualization of entity embedding in HoK and SMAC.

single-agent scenario. Then we look back at Fig.5 and Fig.8. We can see that M^{NS} can outperform M^{NG} in the held-out scenarios. The results in this subsection highlight the benefit of MAGENTA’s rapid fine-tuning to new scenarios across different tasks and the number of agents or entities.

5.5 How does MAGENTA scale with model size?

We obtain the final performance of M^{NS} on all games and choose M^{Exp} as the baseline to compute the normalized score. In Fig.9 It shows that when the size of the model increases, in the complex game, i.e., HoK, MAGENTA has better performance. In less complex games, MAGENTA shows a flat or decreasing performance with

increasing model size. We interpret this result as overparameterization, where a richer model is more fitting than necessary. In fact, we conducted a confirmatory experiment using just a few FC layers in SMAC and NMMO. It can show a performance similar to M^{Exp} .

5.6 What does MAGENTA attend to?

We find that MAGENTA consistently attends to related entities. Fig.10 visualizes the selected attention heads and layers for HoK 3v3 scenarios. By fixing one frame in a running competition in HoK, we find that heads attend to entities such as monster, enemy, and friend, which are the entities highly related to the current agent’s state. The visualization of MAGENTA in SMAC is shown in Appendix 11. We find that heads attend to allies.

5.7 What the learned embedding in games look like?

In this subsection, we visualize the embedded entities in HoK and SMAC using t-SNE [24]. We run MAGENTA for one episode in HoK and collect the embedding of entities for 6000 game frames. Also, we run MAGENTA for 50 episodes in SMAC and collect the embedding of the entities. We can see a clear separation between allies’ embedding and enemies’ embedding in both HoK and SMAC. Even though HoK and SMAC are two different games, MAGENTA can still learn some general knowledge across different games.

6 Conclusion

In this paper, we have made an attempt to develop highly capable generalist agents. Namely, we provide a perspective: viewing different games as different languages. When different tokenizers

are trained for various games, entities are split into tokens. The utilization of transformer models can deal with different feature inputs and various numbers of entities. The attention mechanism in the transformer also encourages the agent to focus on highly related entities. Our results exhibit a clear benefit of using large transformer-based models from an entity perspective in multi-agent multi-game domains. We believe the trends suggest clear paths for future work, that with larger models and larger suites of tasks.

Limitations. We acknowledge some limitations of our conclusions. First, our results are largely based on performance in battle games, such as real-time strategy games, MOBA games, and MMO games, where the rules are similar to each other. Second, the tokenizer is not general for all games. We still need to train specific tokenizers to handle different entities in various games. Third, we still follow the online fashion, which remains the question of whether offline datasets can help boost training efficiency. Lastly, it remains unclear whether we can observe other forms of generalization, such as zero-shot adaptation, as well as whether our conclusions hold for other settings.

References

- [1] Hangbo Bao, Li Dong, and Furu Wei. 2021. Beit: Bert pre-training of image transformers. *arXiv preprint arXiv:2106.08254* (2021).
- [2] Richard Bellman. 1957. A Markovian. *Journal of Mathematics and Mechanics* (1957), 679–684.
- [3] Daniel S Bernstein, Robert Givan, Neil Immerman, and Shlomo Zilberstein. 2002. The complexity of decentralized control of Markov decision processes. *Mathematics of Operations Research* 27, 4 (2002), 819–840.
- [4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33 (2020), 1877–1901.
- [5] Kai-Wei Chang, Akshay Krishnamurthy, Alekh Agarwal, Hal Daumé III, and John Langford. 2015. Learning to search better than your teacher. In *International Conference on Machine Learning*. PMLR, 2058–2066.
- [6] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. *Advances in Neural Information Processing Systems* 34 (2021), 15084–15097.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [9] Yang Gao, Huazhe Xu, Ji Lin, Fisher Yu, Sergey Levine, and Trevor Darrell. 2018. Reinforcement learning from imperfect demonstrations. *arXiv preprint arXiv:1802.05313* (2018).
- [10] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, et al. 2018. Deep q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [11] Max Jaderberg, Wojciech M Czarnecki, Iain Dunning, Luke Marris, Guy Lever, Antonio Garcia Castaneda, Charles Beattie, Neil C Rabinowitz, Ari S Morcos, Avraham Ruderman, et al. 2019. Human-level performance in 3D multiplayer games with population-based reinforcement learning. *Science* 364, 6443 (2019), 859–865.
- [12] Michael Janner, Qiyang Li, and Sergey Levine. 2021. Offline reinforcement learning as one big sequence modeling problem. *Advances in Neural Information Processing Systems* 34 (2021), 1273–1286.
- [13] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).
- [14] Kuang-Huei Lee, Ofir Nachum, Mengjiao Yang, Lisa Lee, Daniel Freeman, Winnie Xu, Sergio Guadarrama, Ian Fischer, Eric Jang, Henryk Michalewski, et al. 2022. Multi-Game Decision Transformers. *arXiv preprint arXiv:2205.15241* (2022).
- [15] Yinhan Liu, Jiatao Gu, Naman Goyal, Xian Li, Sergey Edunov, Marjan Ghazvininejad, Mike Lewis, and Luke Zettlemoyer. 2020. Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics* 8 (2020), 726–742.
- [16] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529–533.
- [17] OpenAI. 2019. OpenAI Five. <https://openai.com/blog/openai-five/>. Accessed March 4, 2019.
- [18] Emilio Parisotto, Jimmy Lei Ba, and Ruslan Salakhutdinov. 2015. Actor-mimic: Deep multitask and transfer reinforcement learning. *arXiv preprint arXiv:1511.06342* (2015).
- [19] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yuri Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. 2022. A generalist agent. *arXiv preprint arXiv:2205.06175* (2022).
- [20] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. 2020. Self-supervised graph transformer on large-scale molecular data. *Advances in Neural Information Processing Systems* 33 (2020), 12559–12571.
- [21] Andrei A Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell. 2015. Policy distillation. *arXiv preprint arXiv:1511.06295* (2015).
- [22] Simon Schmitt, Jonathan J Hudson, Augustin Zidek, Simon Osindero, Carl Doersch, Wojciech M Czarnecki, Joel Z Leibo, Heinrich Kuttler, Andrew Zisserman, Karen Simonyan, et al. 2018. Kickstarting deep reinforcement learning. *arXiv preprint arXiv:1803.03835* (2018).
- [23] Ikehukwu Uchendu, Ted Xiao, Yao Lu, Banghua Zhu, Mengyuan Yan, Joséphine Simon, Matthew Bennice, Chuyuan Fu, Cong Ma, Jiantao Jiao, et al. 2022. Jump-Start Reinforcement Learning. *arXiv preprint arXiv:2204.02372* (2022).
- [24] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
- [25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* 30 (2017).
- [26] Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. 2022. Multi-Agent Reinforcement Learning is a Sequence Modeling Problem. *arXiv preprint arXiv:2205.14953* (2022).
- [27] Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A Massively Multilingual Pre-trained Text-to-Text Transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 483–498.
- [28] Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. 2020. Mastering complex control in moba games with deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 6672–6679.
- [29] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J Kim. 2019. Graph transformer networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [30] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. 2017. Deep Sets. In *Advances in Neural Information Processing Systems*, I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30.